

Legal Clause Classification

Project 0 - Experiment Report

Experiment date: September 14, 2026
Status: Local comparison completed; final-data and API evaluation pending.

1. Objective and Scope

Given one legal contract clause and the complete category catalogue, return exactly one allowed category name. The experiment compares three local Ollama models under the same prompt and scoring rules. It replaces the original binary Yes/No task, which supplied the candidate label.

Example: "Either party may terminate this Agreement without cause upon thirty days' written notice."
Expected label: Termination for Convenience.

This is a synthetic development benchmark, not evidence of real-world legal accuracy.

2. Data and Labeling

The dataset contains **50 synthetic clauses across 25 categories**, with two examples per category. Each item has an ID, clause, expected label, and source. Definitions are loaded from `category_descriptions.csv`; the selected category names are maintained in `src/prompt.py`.

Metadata and extraction fields, including parties, dates, governing law, and warranty duration, are excluded. Clauses express one intended provision. When a specialized license provision overlaps a generic grant, the most specific expressed category takes precedence. This convention simplifies a domain that is often multilabel in practice.

Validation checks the minimum item count, unique IDs, nonempty clauses, normalized duplicate clauses, valid expected labels, and at least two items per tested category. The current labels have not undergone independent two-person review. See Labeling policy.

3. Models and Protocol

Experiment role	Ollama tag	Parameters	Quantization
Baseline	llama3.1:latest	8.0B	Q4_K_M
Top-local replacement	llama3.2:3b	3.2B	Q4_K_M
Cheap-local replacement	llama3.2:1b	1.2B	Q8_0

The replacement roles temporarily occupy the planned API comparison slots. They do not imply API-equivalent quality or cost. The 3B replacement is smaller than the 8B baseline, and the experiment varies quantization as well as model size.

Each model receives the same complete catalogue, definitions, and clause. Requests are sequential, with temperature 0, a 24-token output limit, a 180-second request timeout, and one call per item. There are no retries, warm-up calls, external judges, or prompt changes during the run.

The parser trims surrounding whitespace and accepts only an exact, case-sensitive category name. Prefixes, extra punctuation, explanations, unknown labels, errors, and timeouts count as wrong.

4. Measurement and Hardware

Accuracy is correct predictions divided by all items. Latency is measured from sending a request until its full response arrives; failed requests retain elapsed time. The p50 and p95 values use linear interpolation over ordered latencies.

Generation throughput is the sum of generated tokens divided by the sum of Ollama generation durations, converted from nanoseconds to seconds. Only matched available measurements are used. Requests/hour is based on elapsed time for each model's sequential benchmark loop, including result-writing overhead.

The recorded host runs Windows 11, with 16 logical CPUs and a detected NVIDIA GeForce RTX 4060 Laptop GPU reporting 8188 MiB VRAM. Total RAM and the detailed CPU model were unavailable to automatic detection. GPU detection does not itself establish inference placement. Power draw and hourly cost were not measured.

5. Measured Results

Source run: 20260914T084039794325Z. All **150 classification calls** completed.

Model	Correct	Accuracy	Parse errors	Timeouts	Server errors
llama3.1:latest	50/50	100%	0	0	0
llama3.2:3b	39/50	78%	2	0	0
llama3.2:1b	5/50	10%	0	0	0

Model	p50 (ms)	p95 (ms)	Generation tok/s	Requests/hour
llama3.1:latest	2332.31	2491.51	30.42	1413.93
llama3.2:3b	2251.68	2469.24	42.53	1491.06
llama3.2:1b	2189.76	2470.40	42.95	1364.01

First-request latencies were 12.97, 9.44, and 20.52 seconds respectively. Model loading and prompt caching affect the comparison; the measured request rate is not a steady-state capacity estimate. Faster token generation did not produce a similarly large end-to-end latency improvement.

The earlier standalone multiclass baseline also scored 50/50 and remains separately preserved. Its timings are not mixed into this table. The old binary result is not part of this experiment.

6. Wrong-Answer Analysis

The 8B baseline made no mistakes on this small synthetic set. This does not establish generalization.

The 3B model made nine incorrect allowed-label predictions and two parse errors. Representative cases:

Item	Expected	Returned output
012	Termination for Convenience	No-Solicit of Customers
017	Revenue/Profit Sharing	Price Restrictions
025	IP Ownership Assignment	Non-IP Ownership Assignment
029	License Grant	Non-Transferable License

Item	Expected	Returned output
030	License Grant	Exclusivity
046	Cap on Liability	Time Limit

Items 025 and 046 returned labels outside the catalogue. Both were rejected. The license-grant errors illustrate confusion between a general permission and a narrower contractual restriction.

The 1B model made 45 incorrect allowed-label predictions. It repeatedly chose Non-Compete or Non-Transferable License for unrelated clauses. Zero parse errors therefore did not indicate semantic accuracy. The observations suggest poor performance under this catalogue and prompt; they do not prove the cause.

7. Cost and Model Choice

Self-hosted cost is **unavailable** because `LOCAL_HARDWARE_COST_PER_HOUR_USD` was not supplied. No monetary result is invented.

Cost per 1,000 requests = hourly hardware cost / measured requests per hour * 1,000.

Cost at 100x benchmark traffic = cost per 1,000 * 5 for this 50-item dataset.

The calculation assumes the same sequential rate and hourly cost at 5,000 requests. It does not model concurrency, utilization, scaling, API pricing, or break-even.

Retain **llama3.1:latest** for this development task. It achieved the highest measured accuracy with only a small median-latency disadvantage. The 3B and 1B models remain useful comparison baselines. A deployment decision requires independent real-clause evaluation and, when implemented, actual API comparisons.

8. Limitations and Remaining Work

Replace or augment synthetic clauses with independently sourced real examples and documented provenance.

Have two people independently verify every final label and resolve disagreements.

Evaluate more examples per category and explicitly handle genuinely overlapping or out-of-catalogue clauses.

Repeat measurements with a documented startup/cache protocol before drawing capacity conclusions.

Record missing hardware details, power measurements, and the provenance of any hourly-cost assumption.

Implement the actual top and cheap API comparison later; current local replacements do not fulfill that comparison.

Historical runs lack a clause snapshot; the dashboard uses current dataset text for those runs. Future runs retain their own dataset and prompt catalogue.

9. Reproduction and Evidence

```
python src/run.py --validate-only
python src/run.py --compare
python src/dashboard.py --port 8080
```

The dashboard is read-only and visualizes saved CSVs. It does not call models or rescore answers.

Comparison CSV

Model digests and metadata

Baseline predictions

3B predictions

1B predictions

Per-model hardware notes and summaries sit beside the prediction files. Markdown is the editable source; the PDF exporter reads this document directly.